

机器学习与数据挖掘/第一周

姓名	学号
汪宣彤	22336221

1. 描述什么是线性回归，并给出一个实际应用的例子

1) 什么是线性回归？

线性回归是一种用于分析和预测两种或多种变量之间关系的统计方法。它通过拟合一条直线，表示输入变量和输出变量之间的关系

线性回归的数学表达式如下：

$$y = \sum_{i=1}^n w_i x_i + w_0$$

其中 y 是预测值， x_i 是输入值， w_i 是权重， w_0 是偏置项

线性回归的核心思想是找到最优的 w_i 和 w_0 使得模型预测的 y 值与实际值之间的误差最小

2) 实际应用的例子：房价预测

假设根据房屋的面积和房屋的地段来预测房价。其中：

- 自变量 x_1 是房屋的面积
- 自变量 x_2 是房屋的地段
- 因变量 y 是房屋的价格

线性回归模型会找到一个线性方程来描述面积和地段与价格的关系。例如：

$$y = 0.5x_0 + 0.3x_1 + 30$$

通过这个模型，能够根据不同的面积快速估算房价，从而提供有价值的参考

2. 形式化地描述如何通过最小化损失函数来找到最佳的模型参数

1) 定义模型

假设有一个机器学习模型，其目标是通过输入数据 X 预测目标变量 y 。该模型由参数 w 控制， f 是模型的预测函数， $\hat{y}$ 是模型对目标变量 y 的预测值

则模型的输出可以表示为：

$$\hat{y} = f(X, w)$$

2) 定义损失函数

为了衡量模型预测 $\hat{y}$ 和真实值 y 之间的差异，引入**损失函数**，记为 $J(w)$ ，其作用是根据模型的预测值和真实值的差异来衡量模型的性能。常见的损失函数是**均方误差**（MSE）：

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

其中：

- m 是样本的数量
- y_i 是第 i 个样本的真实值
- $\hat{y}_i = f(X_i, w)$ 是第 i 个样本的预测值

损失函数 $J(w)$ 反映了模型参数 w 如何影响预测误差。**目标是找到使损失函数最小化的参数 w** ，即找到最佳模型参数

3) 梯度下降法和计算梯度

此处使用**梯度下降法**最小化损失函数。梯度下降法是迭代优化算法，通过不断调整模型参数 w ，沿着损失函数的负梯度方向移动，逐步接近损失函数的最小值。

梯度是损失函数的偏导数向量，对于每个模型参数 w_j ，梯度的计算公式为：

$$\frac{\partial J(w)}{\partial w_j} = -\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i) \cdot x_{ij}$$

其中 x_{ij} 是第 i 个样本的第 j 个特征值

4) 更新模型参数

根据梯度的计算结果，使用梯度下降公式更新每个参数 w_j ：

$$w_j := w_j - \alpha \frac{\partial J(w)}{\partial w_j}$$

通过多次迭代，不断更新参数 w ，使得损失函数逐渐减少，最终接近其最小值

5) 收敛条件

梯度下降法会持续迭代更新参数，直到满足某些收敛条件，例如：

- 梯度接近 0
- 损失函数的变化量极小

6) 最佳模型参数

最终，梯度下降法找到的模型参数 w^* 满足：

$$w^* = \arg \min_w J(w)$$

即参数 w^* 使损失函数达到最小值。此时，模型的预测误差最小，得到的 w^* 是模型的最佳参数

3. 解释梯度在向量或矩阵上的意义，并讨论它在优化问题中的作用

1) 梯度在向量或矩阵上的意义

在多变量函数中，梯度是一个向量，包含了该函数对所有输入变量的偏导数

- **标量函数**（函数输出数值）

对于函数 $f(x_1, x_2, \dots, x_n)$ ，其梯度是一个包含所有偏导数的向量：

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

- **向量函数**（函数输出向量）

如果函数 $f(x)$ 的输出是一个向量，则梯度为一个矩阵，其元素是函数的偏导数：

$$\frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

其中， m 是函数的输出维度， n 是输入维度

2) 梯度在优化问题中的作用

在优化问题中，梯度的作用是提供有关目标函数在当前点变化的方向和速率的信息

- **方向指示**

梯度向量的方向指示了函数在当前点上升最快的方向。在优化中，我们希望沿着梯度的反方向（下降最快的方向）更新参数，以最小化目标函数

- **速率指示**

梯度的大小表示函数在当前点的变化速率。梯度较大的地方意味着函数的变化较快，梯度较小的地方变化较慢。优化算法可以利用梯度的大小来调整更新步长（即学习率）

4. 描述梯度下降法的工作原理，并讨论学习率对模型收敛速度的影响

1) 梯度下降法的工作原理

- 初始化参数

首先随机初始化模型参数，参数可以随机生成或设定为0

- 计算损失函数

根据当前参数计算损失函数 $J(w)$

- 计算梯度

计算损失函数 $J(w)$ 对模型参数的梯度 $\nabla J(w)$ 。梯度向量中的每个元素是损失函数对相应参数的偏导数，反映了参数在该点对损失函数的影响。梯度的反方向表示损失函数下降最快的方向

- 参数更新

根据梯度的反方向更新参数：

$$w := w - \alpha \nabla J(w)$$

其中， α 是**学习率**，用于控制每次更新的步长

- 重复迭代

重复上述过程，即每次迭代更新参数并重新计算损失函数，直到达到某个停止条件

2) 学习率对模型收敛速度的影响

学习率 (α) 是梯度下降法中的重要参数，它决定了每次更新时步长的大小

- 学习率过大

- 现象：如果学习率太大，参数更新的步伐会过大，迭代可能发散
- 结果：模型不收敛，甚至可能损失函数越来越大，无法找到最优解

- 学习率过小

- 现象：如果学习率太小，参数每次更新的步伐过小，收敛速度极慢，训练时间会大幅增加
- 结果：模型收敛非常慢，训练过程可能需要很多次迭代才能接近最优解

◦ 适当的学习率

如果学习率选得适当，模型可以在合理的时间内快速朝最优点收敛。参数更新每一步都有效地减小了损失函数，使得模型在较少的迭代次数内接近最优解

因此，设置合适的学习率非常重要

5. 解释随机梯度下降与传统梯度下降的区别，并讨论其在处理大数据集时的优势

1) 随机梯度下降与传统梯度下降的区别

	传统（批量）梯度下降	随机梯度下降
梯度计算	基于整个数据集	基于一个样本
每次更新的开销	大，需要计算所有样本的梯度	小，只需计算一个样本的梯度
收敛速度	需要更长时间，因为每次更新计算成本高	每次更新都很快，但收敛可能不稳定
适用场景	小数据集	大数据集，实时数据训练

2) 随机梯度下降在处理大数据集时的优势

◦ 开销小，效率高

- 批量梯度下降需要在每次更新时遍历整个数据集，计算所有样本的梯度，计算量大且耗时。对于超大规模的数据集，每次迭代都非常缓慢，且需要大量的内存
- 随机梯度下降每次只处理一个样本，因此可以在有限的内存中处理超大规模的数据集

◦ 加快收敛

由于随机梯度下降每次更新参数的频率高，它能够比批量梯度下降更快地靠近最优解。即便它在收敛过程中会出现一定的波动，但通过更频繁的更新，它可以更快地找到接近最优的解

◦ 更好地跳出局部最优

批量梯度下降由于每次更新的方向非常精确，往往会陷入局部最优。而随机梯度下降由于每次更新依赖于单个样本的梯度，更新方向具有随机性，因此有时能跳出局部最优，更接近全局最优

6. 讨论在随机梯度下降中，梯度的估计值与真实梯度之间的关系

1) 真实梯度

在传统的批量梯度下降中，真实梯度是基于整个训练数据集的损失函数计算得到的梯度值，代表全局的梯度方向：

$$J(w) = \frac{1}{m} \sum_{i=1}^m \nabla J(w; x^{(i)}, y^{(i)})$$

其中， m 是数据集的样本总数， $\nabla J(w; x^{(i)}, y^{(i)})$ 是每个样本对损失函数的贡献

2) 梯度估计值

在随机梯度下降中，梯度的估计值是基于**单个样本**或**一个小批量样本**计算得到的，使用它来更新模型参数：

$$J(w) \approx \nabla J(w; x^{(i)}, y^{(i)})$$

与真实梯度相比，随机梯度下降中每次迭代并不考虑所有样本，**因此，估计梯度相当于真实梯度的一个近似值，包含更多随机性和噪声，更新方向不如真实梯度稳定**

7. 列出并简要描述至少三种除了梯度下降之外的优化方法，并讨论它们的优缺点

1) 牛顿法

- 牛顿法是一种基于二阶导数的优化方法。它通过近似目标函数的二次曲线，在每一步利用二阶信息来更新参数，从而更快地找到局部最优解
- 优点
 - **收敛速度快**：由于利用了二阶信息，牛顿法通常可以在较少的迭代次数内收敛到最优解，尤其是在问题接近二次曲线时
 - **精确性高**：能够直接计算步长，无需手动选择学习率，使得每次更新更精准
- 缺点
 - **不适合高维问题**：当参数空间维度过高时，牛顿法的计算开销和内存需求都急剧增加，因此不适合处理大规模问题

2) 共轭梯度法

- 共轭梯度法是梯度下降的改进版，不仅利用当前梯度的方向，还结合了之前梯度方向的信息，使得每次迭代都朝着共轭方向前进，从而加速收敛
- 优点
 - **适合高维问题**：与牛顿法不同，共轭梯度法不需要计算二阶导数，适合维度较高的优化问题
 - **内存需求低**：由于它是一种基于向量的迭代方法，对存储和计算资源的需求比牛顿法低，适合大规模问题
- 缺点
 - **只适用于二次优化问题**：它主要用于线性或二次优化问题，对于其他复杂的非线性问题不太适用

3) 拟牛顿法

- 拟牛顿法是牛顿法的变体，它不直接计算牛顿法需要的 Hessian 矩阵，而是通过迭代更新的方式近似 Hessian 矩阵，它逐步更新和逼近二阶导数的信息，从而进行更有效的优化
- 优点
 - **更高效**：拟牛顿法避免了显式计算 Hessian 矩阵的高昂代价，适用于高维问题
 - **收敛快**：虽然比牛顿法慢，但比梯度下降快，尤其是在接近最优解时表现很好

- 缺点

- **内存需求较大**：虽然避免了直接计算 Hessian，但仍需存储和更新矩阵，内存需求较大
- **实现复杂**：相较于简单的梯度下降法，拟牛顿法的实现较为复杂

4) 协调下降法

- 协调下降法每次只更新一个变量（参数），而保持其他变量固定。然后在所有变量上依次循环，逐步减少损失函数的值。它通过逐坐标优化的方式，逐渐接近全局最优解

- 优点

- **计算效率高**：每次只需要计算一个变量的更新，因此计算开销较小
- **简单易实现**：算法相对简单

- 缺点

- **收敛速度慢**：在高维问题中，逐个变量更新的方式收敛较慢